

DSA Advanced

Placement Training Program

Trees & Graphs

A tree is a hierarchical structure of nodes with no cycles, where each node (except the root) has exactly one parent — binary trees (each node has at most two children) and binary search trees (left subtree values smaller, right subtree values larger, enabling fast search) are the most commonly tested variants. A graph generalizes further, allowing any node to connect to any other node, with or without cycles, directed or undirected — representing anything from social networks to road maps to dependency chains. Traversal is the fundamental operation on both: depth-first search (DFS, using recursion or an explicit stack) explores as far as possible down one path before backtracking; breadth-first search (BFS, using a queue) explores level by level, which is what makes it the right choice whenever you need the shortest path in an unweighted graph.

Dynamic Programming

Dynamic programming (DP) solves problems by breaking them into overlapping subproblems, solving each subproblem once, and reusing (rather than recomputing) that result whenever it's needed again — either by storing results in a table as you go (bottom-up/tabulation) or by caching recursive calls' results (top-down/memoization). The signal that a problem might need DP: a naive recursive solution that recomputes the exact same subproblem many times (classic example: naive recursive Fibonacci recalculates `fib(3)` repeatedly across different branches of the recursion tree). DP is widely considered the hardest interview topic precisely because recognizing the subproblem structure of a new problem takes real practice — the individual technique, once recognized, is usually mechanical.

Greedy Algorithms

A greedy algorithm builds a solution by always making the choice that looks best at the current step, without reconsidering that choice later — and for some problems, this locally-optimal approach provably produces a globally optimal result (classic examples: activity selection, certain scheduling problems, Dijkstra's shortest-path algorithm). The catch is that greedy doesn't work for every problem — proving a greedy approach is actually correct for a given problem (rather than just producing a plausible-looking answer that happens to be wrong on some input) is itself a skill, and when in doubt, it's worth testing a greedy solution against a brute-force one on small examples before trusting it.

Sorting/Searching at Scale

Beyond knowing that sorting algorithms exist, understanding their tradeoffs matters: merge sort and heapsort guarantee $O(n \log n)$ in all cases but merge sort needs $O(n)$ extra space; quicksort is typically faster in practice with good average-case performance but has a worst case of $O(n^2)$ on adversarial input; and for nearly-sorted or small data, simpler $O(n^2)$ algorithms can actually outperform the 'better' $O(n \log n)$ ones due to lower constant overhead. Binary search — repeatedly halving a sorted search space — achieves $O(\log n)$ lookup, but only works on sorted data, which is itself a common reason to sort data upfront even when sorting isn't the end goal.

Mock Coding Tests

Timed mock tests mirroring real placement coding rounds are where the fundamentals and advanced topics get pressure-tested together under realistic constraints. Beyond solving the problem correctly, mock tests build the specific skill of managing time across multiple problems — recognizing quickly when an approach isn't working and needs to be abandoned, rather than sinking the whole time budget into one problem. Reviewing wrong or slow answers afterward, specifically identifying whether the gap was conceptual (didn't know the technique) or execution (knew the technique but made an implementation mistake), is what actually improves performance round over round — simply doing more problems without that review tends to plateau quickly.

Learn More

NeetCode — DSA Roadmap & Practice

<https://neetcode.io/>

GeeksforGeeks — Data Structures & Algorithms

<https://www.geeksforgeeks.org/data-structures/>

Visualgo — Visualizing Data Structures & Algorithms

<https://visualgo.net/>